
ИВМиМГ СО РАН

ул. Лаврентьева, д. 6
Новосибирск, Россия

Платформа для планирования и проведения экспериментов на цифровом
двойнике ЦКП СКИФ

Control plane

21 октября 2025 г.

ОБЗОР

Control Plane - веб платформа, которая является связующим звеном между пользователями и ядром системы (супервизором). Предполагается, что именно в ней будут храниться данные о пользователях, их полномочия, история экспериментов и логи взаимодействия со всеми элементами системы.

Цель Control Plane - создать гибкую, масштабируемую и высокопроизводительную инфраструктуру, которая позволит ученым инженерам и студентам моделировать сложные физические процессы, тестировать различные сценарии экспериментов и управлять цифровым двойником ускорителя в разных режимах времени.

ЦЕЛИ

1. **Обеспечить структуру прав:** пользователи должны работать в рамках своих отдельных “Рабочих зон” и не мешать друг-другу, при желании можно выдавать права на свои рабочие зоны, чтобы люди тоже могли взаимодействовать с ними
2. **Логирование изменений / история изменений:** на платформе должно сохраняться каждое действие - кто, что, зачем менял, создавал или удалял.
3. **Версионирование:** должно поддерживаться версионирование экспериментов и возможность откатываться назад при желании, возможность сравнивать разные версии.

-
4. **Визуализация:** интеграция с каким-либо визуализатором моделей (нужно обсудить с другим дипломником)
 5. **Time to Market:** популярный термин в коммерческой разработке - в нашем случае я имею ввиду скорость привыкания студентов/персонала к работе с платформой - время от постановки задачи до проведения первого эксперимента должно быть минимальным.
 6. **Аналитика результатов:** Интерпретация результатов экспериментов из супервизора, а также промежуточных этапов

ТЕХНИЧЕСКИЕ ПОДРОБНОСТИ

Сервис будет состоять из frontend, backend частей, а также будет иметь свою бд для хранения информации о пользователях и их правах, логах, версиях и т.д.

Предварительный выбор языков и технологий:

Backend:

- 1) **Выбор языка:** [Golang](#), Причины: простой, быстрый, эффективный, удобен для деплоя, хорошо подходит для микро сервисной архитектуры - также перспективен для поддержки в будущем - в последние годы начал набирать сильную популярность у студентов ФИТа.
- 2) **Выбор фреймворка:** [Gin](#) - на данный момент наиболее поддерживаемый фреймворк для роутинга запросов, покрывает все необходимые кейсы для текущего проекта
- 3) **Обновление состояний:** [gorilla WebSocket](#): предлагается использовать вебсокеты для отображения актуального состояния экспериментов, а не посылать периодические запросы с фронта
- 4) **База данных:** [PostgreSQL](#) - самая часто используемая база данных, обладает всем необходимым функционалом, спокойно сможет выдержать нагрузку до 1000 пользователей без дополнительных оптимизаций. Также крайне выгодна в плане поддержки, если нужны будут необычные доработки - НГУ сотрудничает с Postgres, а также большинство студентов умеют работать с этой базой данных
- 5) **Кеширование:** большая нагрузка не предвидится, думаю здесь будет достаточно [встроенного в Golang фреймворка для кеша](#)

Frontend:

- 1) **Выбор языка:** [TypeScript](#), Причины: строго типизирован (позволяет отлавливать большинство ошибок еще на этапе разработки), активно развивается + на нем уже

есть все необходимые инструменты для развития как самописной 3Д визуализации (в виде [WebGL](#)), так для написания внешней обертки платформы React

- 2) **Выбор фреймворка для платформы:** [React](#) - быстрый в освоение и скорости разработки, отлично подходит для проектов, которым нужно быстро стартовать или для тех, которые будут переписываться в будущем
- 3) **Визуальная составляющая:** [Gravity UI](#), так как в разработке профессиональный дизайнер и фронтендер не будут участвовать - хорошим решением будет завязаться на активно поддерживаемую систему компонентов от Яндекса, что позволит улучшать дизайн (если это будет необходимо), а также упростит проработку на этапе макетов, так как есть готовые компоненты для [Figma](#).
- 4) **Возможный путь интеграции визуализации в платформу** - [написать свой движок на WebGL](#), это позволит стать независимыми в 3Д ПО и позволит неограниченно кастомизировать систему под наши нужды (обсуждаемо). Проблем с интеграцией не будет.
- 5) **Хранение состояний на стороне клиента:** Redux

РЕФЕРЕНСЫ (Open Source)

1. Eclipse Ditto

Что это

Eclipse Ditto - open-source фреймворк для управления цифровыми двойниками IoT-устройств. Он предоставляет абстракцию “текущее / желаемое / заявленное состояние” устройств, API для синхронизации, управление состоянием, поддерживает подключение через WS / HTTP и др.

Также Ditto использует [гранулярную систему прав](#), что позволяет давать права на отдельные компоненты цифрового двойника

Что из этого полезно взять

- 1) Модель текущее / желаемое / заявленное состояние - разделение, которое удобно для управления.
- 2) Гранулярная авторизация доступа к элементам цифрового двойника - может быть полезна, если мы не хотим чтобы пользователи мешали друг-другу и работали только со своими частями.

2. KTWIN

Что это

KTWIN - исследовательская платформа, предлагающая serverless + Kubernetes-подход к развертыванию цифровых двойников. Суть: дать разработчикам возможность описать модель, а система сама поднимает нужные сервисы, масштабирует, управляет инфраструктурой.

Очень похоже на наш подход, но здесь каждая модель состоит из группы сервисов - что снимает бизнес логику с главного сервера. При желании такой подход можно переиспользовать - супервизор по-сути превратиться в оркестратор (сервис, которые занимается поднятием сервисов и сбором их состояния с “агентов” - сервисов, которые поднимаются для каждой модели)

Что из этого полезно взять

- 1) Идея абстракции инфраструктуры.
- 2) Автоматическое масштабирование
- 3) Возможность описания “модели + требования к ресурсам” декларативно и система сама решает, как развернуть (K8s, поды, контейнеры)..

3. Snap4City

Что это

Snap4City - фреймворк / платформа для цифрового двойника города и инфраструктур, ориентированная на real-time интеграцию данных, анализ и “what-if” сценарии. У неё есть модуль сбора данных, аналитика, визуализация, сценарии симуляции и веб-интерфейс.

Полезные идеи

- 1) Компонент “what-if analysis” - возможность изменять параметры и смотреть последствия в симуляции (**Можно использовать динамические данные в конфигурациях моделей, которые пользователь сможет менять не занимаясь передеплоем - долгие передеploi могут стать большой проблемой в будущем**)
- 2) Непрерывное обновление физических / сенсорных данных + реакция на изменения

Что можно использовать в Control Plane?

Функционал	Платформа	Как можно встроить
Состояния цифрового двойника	Eclipse Ditto	Разделять состояния: “реальное состояние” (от симулятора), “целевое состояние” (задача). Это позволяет управлять командными инструкциями и синхронизацией.
Гранулярный доступ	Eclipse Ditto	Использовать ACL не только на уровне экспериментов, но и на уровне параметров / объектов внутри эксперимента.
Абстракция подключения источников данных	Ditto + Snap4City	Ввести свои коннекторы, которые позволят подключать реальные данные к экспериментальным данным
Декларативное описание моделей / развертывания	KTWIN	Пусть эксперимент задаётся декларативно (JSON/YAML с моделью + ресурсами), и система поднимает нужные сервисы.
Авто-масштабирование	KTWIN	Не держать экспериментальные инстансы «вечно» активными - выключать/разгружать, когда нет активности.
динамическая конфигурация / ветвления	Snap4City	В UI дать возможность изменять параметры (например, физические константы, параметры модели) и запускать симуляции ветвлений.

Контейнерная
архитектура /
модульность

Snap4City

Проектировать микросервисы

Мониторинг /
логирование / история
/ версии

Другие платформы

Встроить логи, трассировки,
хранить версии моделей и
записывать состояние
экспериментов.